

Weather Prediction Analytics using Snowflake, Airflow, and dbt

Course: SJSU DATA 226 **Team members:** Yunseo Oh (020746206), Isabella Shi (019256965)

Repository: <https://github.com/yunseoyunnioh/data226-weather-analytics> **Date:** September 26, 2026

1. Problem Statement

Weather affects planning decisions -- commuting, travel, agriculture, retail demand -- but raw daily weather readings (a max/min temperature and a precipitation total) don't answer questions like "is this warmer than usual" or "how long has it been dry." This project builds a pipeline that pulls daily weather for two cities, Seoul and Toronto, ingests it into a cloud data warehouse, derives multi-day analytics from it (moving averages, anomalies, rolling precipitation, dry spell length), and exposes those analytics through a BI dashboard -- so a viewer can compare the two cities' current weather trend at a glance instead of reading raw numbers.

A secondary goal, driven by the assignment itself, was architectural: build **one** pipeline that treats "which city" as configuration/data rather than duplicating the ETL and transformation logic per city.

2. Requirements

1. Ingest daily weather for two cities (Seoul, Toronto) from the Open-Meteo API (<https://api.open-meteo.com/v1/forecast>).
2. Load it into Snowflake with a transaction-safe, idempotent write path (safe to rerun without creating duplicate rows).
3. Orchestrate the ingestion with Apache Airflow, using Airflow Variables for city configuration and an Airflow Connection for Snowflake credentials -- no hardcoded secrets.
4. Transform the raw data with dbt into meaningful, per-city analytics (moving averages, anomaly, rolling precipitation, dry spell length), with schema tests and a snapshot.
5. Trigger the dbt run from Airflow, strictly after the ETL succeeds.
6. Visualize the resulting analytics in a BI tool (Superset, Preset, or Tableau) with at least two screenshots showing the dashboard responding to a filter change.
7. Ship one GitHub repository containing all code, with DAGs under `dags/` and the dbt project under `dbt/` , plus a README and this report.

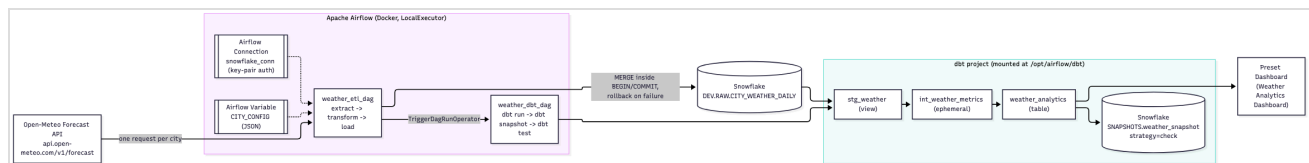
3. Specifications

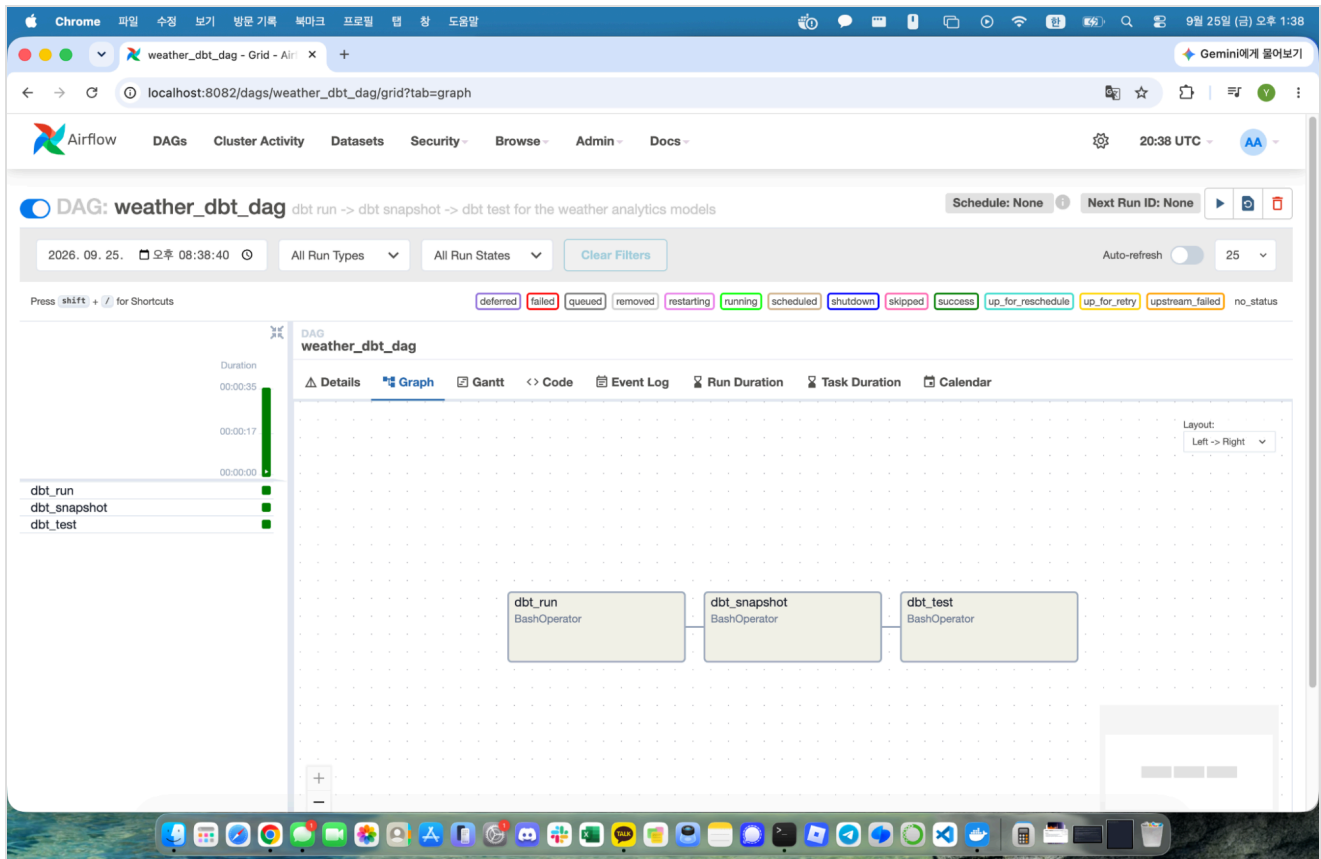
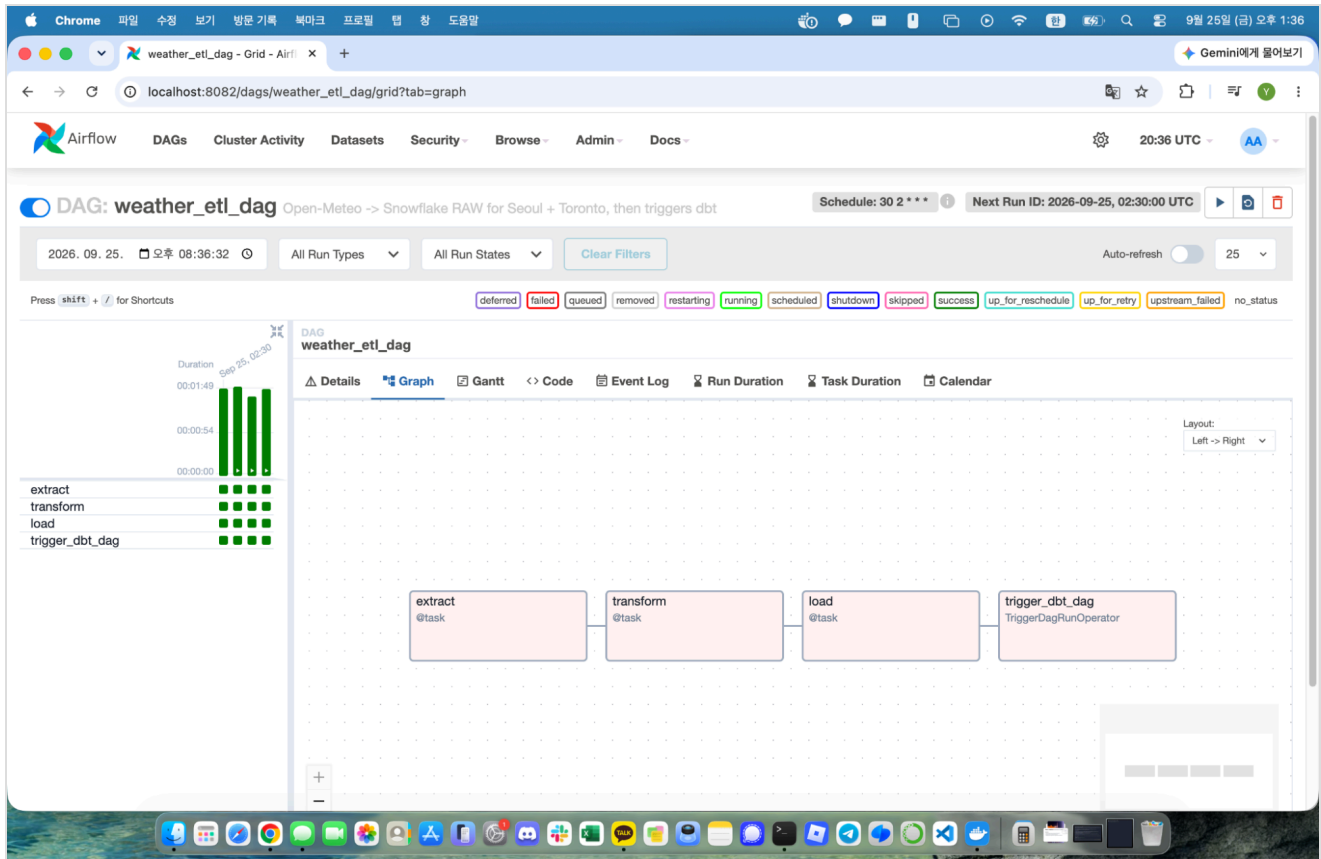
Component	Choice	Why
Source API	Open-Meteo /v1/forecast , past_days=60 , forecast_days=1	Required endpoint; past_days gives enough trailing history in one request per city for 7-day windows and an anomaly baseline
City config	Airflow Variable CITY_CONFIG (JSON: name -> lat/lon/timezone)	Adding a city means adding one JSON entry, no code change
Orchestration	Two DAGs: weather_etl_dag (schedule) -> weather_dbt_dag (triggered, schedule=None)	Makes the ETL-before-dbt dependency explicit and inspectable in the Airflow UI, not just implied by cron timing
Raw table	DEV.RAW.CITY_WEATHER_DAILY , PK (city, date)	New table name, distinct from the single-city HW2/HW3 WEATHER_DAILY table, to avoid touching already-graded data with an incompatible schema
Idempotency	MERGE keyed on (city, date) inside BEGIN / COMMIT , with a duplicate-key check before COMMIT	Snowflake does not enforce declared primary keys on standard tables, so uniqueness is enforced by the merge logic and verified explicitly, not assumed from the DDL
dbt layers	stg_weather (view) -> int_weather_metrics (ephemeral) -> weather_analytics (table)	Mirrors the layering used in the team's HW4 dbt project
Snapshot	weather_snapshot , strategy='check' on weather_analytics	No natural updated_at column exists, and Open-Meteo revises very recent days as more observations arrive, so check is more appropriate than timestamp
BI tool	Preset (cloud-hosted Apache Superset)	Free tier, no local install/Docker needed, connects directly to Snowflake

4. Architecture

Full Mermaid source and narrative notes: docs/architecture.md . Rendered diagram:

System Architecture Diagram. Open-Meteo -> Airflow ETL DAG -> Snowflake RAW -> (triggered) Airflow dbt DAG -> dbt staging/intermediate/analytics -> Snowflake analytics layer -> Preset dashboard, with the CITY_CONFIG Variable and snowflake_conn Connection feeding the ETL DAG and the weather_snapshot snapshot sitting alongside the analytics table.





5. Data Flow

1. `weather_etl_dag.extract()` reads `CITY_CONFIG`, calls Open-Meteo once per city, returns raw JSON payloads.
2. `weather_etl_dag.transform()` flattens each city's daily arrays into `(city, latitude, longitude, date, temp_max, temp_min, precipitation, weather_code)` rows, validating array lengths and rejecting an empty batch.
3. `weather_etl_dag.load()` upserts those rows into `DEV.RAW.CITY_WEATHER_DAILY` via a staged `MERGE` inside one transaction, then triggers `weather_dbt_dag`.
4. `weather_dbt_dag` runs `dbt run` (builds `stg_weather`, `int_weather_metrics`, `weather_analytics`), then `dbt snapshot` (`weather_snapshot`), then `dbt test`.
5. The BI tool queries `DEV.ANALYTICS.WEATHER_ANALYTICS` directly.

6. Airflow Implementation

- **DAG 1 -- `weather_etl_dag`** (`days/weather_etl_dag.py`): TaskFlow API (`@task`), tasks `extract` → `transform` → `load`, scheduled daily (`30 2 * * *`), `catchup=False`, `max_active_runs=1`. Final task `trigger_dbt_dag` (`TriggerDagRunOperator`) starts `weather_dbt_dag` after `load` succeeds.
- **DAG 2 -- `weather_dbt_dag`** (`days/weather_dbt_dag.py`): three `BashOperator` tasks, `dbt_run` → `dbt_snapshot` → `dbt_test`, each invoking `dbt <command> --profiles-dir /opt/airflow/dbt` against the dbt project mounted into the container. `schedule=None`: this DAG never runs on its own timer, only via the trigger from DAG 1.
- **Airflow Variable:** `CITY_CONFIG`, a JSON object mapping city name to `{latitude, longitude, timezone}`.
- **Airflow Connection:** `snowflake_conn`, type `Snowflake`, key-pair authentication (private key file mounted read-only into the container; passphrase stored only in the Connection's Password field / a local `.env`, never in code).
- **Exception handling:** `load()` wraps the transaction in `try/except/finally`; on any exception it issues `ROLLBACK`, logs the outcome, and **re-raises** so the Airflow task is correctly marked failed (no silent swallowing). `extract()` / `transform()` raise `ValueError` on malformed API responses or empty batches rather than loading partial data.

Chrome | 파일 | 수정 | 보기 | 방문 기록 | 북마크 | 프로필 | 탭 | 창 | 도움말 | 9월 25일 (금) 오후 1:35

List Variable - Airflow

localhost:8082/variable/list/

Airflow | DAGs | Cluster Activity | Datasets | Security | Browse | Admin | Docs | 20:35 UTC | AA

파일 선택 | 선택된 파일 없음 | ☒ Overwrite if exists | ☐ Fail if exists | ☐ Skip if exists | Import Variables

List Variable

Search

+ Actions

Record Count: 1

	Key	Val	Description	Is Encrypted
☐	CITY_CONF...	{"Seoul": {"latitude": 37.5665, "longitude": 126.9780, "timezone": "Asia/Seoul"}, "Toronto": {"latitude": 43.6...		False

Version: v2.10.1
Git Version: .release:854173176f372f6509800ed446286c32cb75045e

Chrome | 파일 | 수정 | 보기 | 방문 기록 | 북마크 | 프로필 | 탭 | 창 | 도움말 | 9월 25일 (금) 오후 1:34

List Connection - Airflow

localhost:8082/connection/list/

Airflow | DAGs | Cluster Activity | Datasets | Security | Browse | Admin | Docs | 20:34 UTC | AA

List Connection

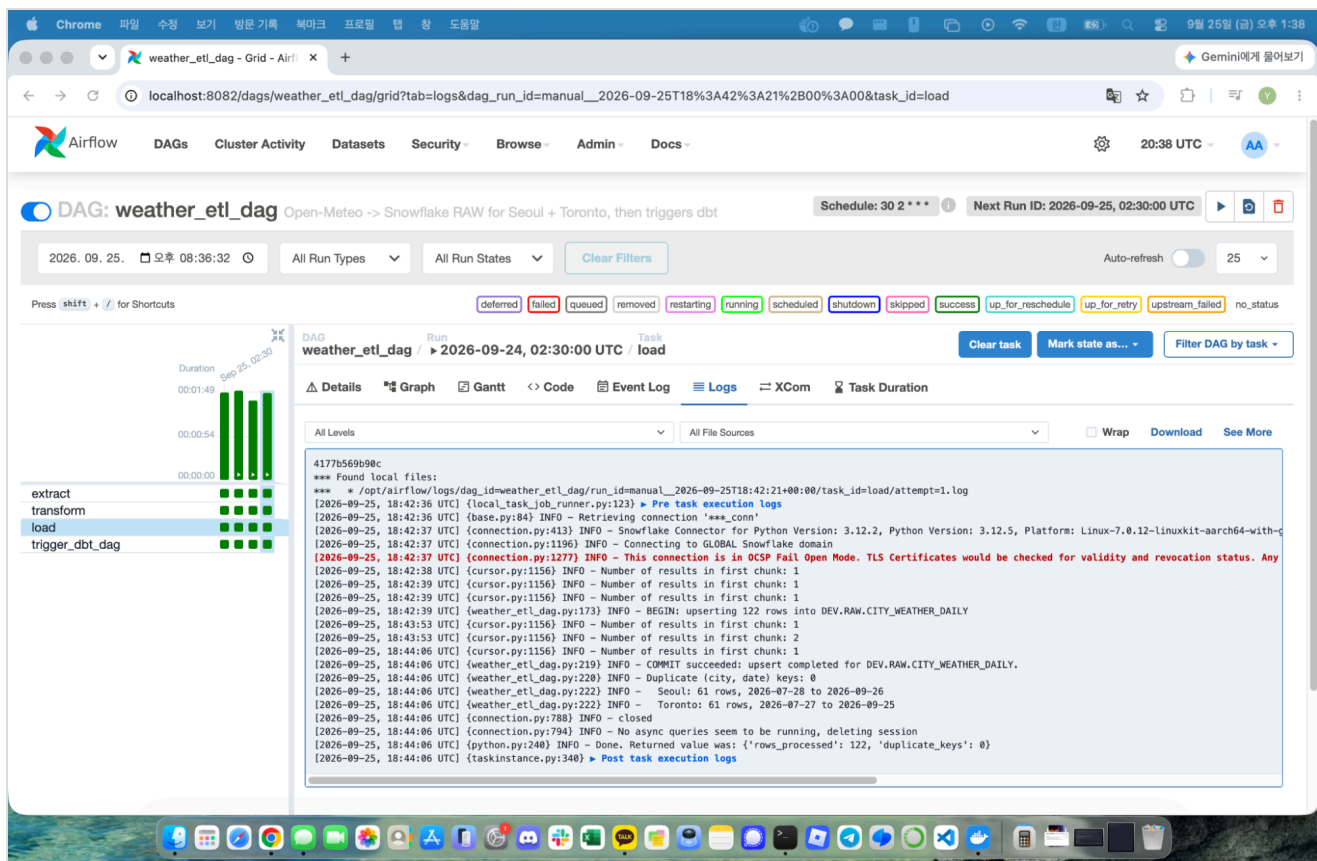
Search

+ Actions

Record Count: 1

	Conn Id	Conn Type	Description	Host	Port	Is Encrypted	Is Extra Encrypted
☐	snowflake_conn	snowflake				False	False

Version: v2.10.1
Git Version: .release:854173176f372f6509800ed446286c32cb75045e



7. Snowflake Design

Database `DEV`, schema `RAW` for the landing table, schema `ANALYTICS` for dbt's output, schema `SNAPSHOTS` for the dbt snapshot. See `sql/create_tables.sql` for the raw table DDL (also embedded in `weather_etl_dag.py`'s `load()` so the pipeline is self-provisioning).

8. Table Structures

All four tables below were inspected directly in the live Snowflake account (`DESC TABLE`) to confirm actual column types and nullability, not assumed from the DDL/model SQL alone -- Snowflake sometimes normalizes or relaxes both (e.g. `INTEGER` is stored as `NUMBER(38,0)`; `CREATE TABLE AS SELECT` makes every output column nullable regardless of the source).

DEV.RAW.CITY_WEATHER_DAILY (raw, loaded by Airflow)

Column	Type	Nullable	Constraint	Description
CITY	VARCHAR(64)	N	part of PK	City name (Seoul , Toronto)
LATITUDE	FLOAT	N	--	Requested latitude
LONGITUDE	FLOAT	N	--	Requested longitude
DATE	DATE	N	part of PK	Local calendar date of the reading
TEMP_MAX	FLOAT	Y	--	Daily max temperature, Celsius
TEMP_MIN	FLOAT	Y	--	Daily min temperature, Celsius
PRECIPITATION	FLOAT	Y	--	Daily precipitation total, mm
WEATHER_CODE	NUMBER(38,0)	Y	--	WMO weather interpretation code (declared INTEGER ; Snowflake stores all integers as NUMBER(38,0))
LOADED_AT	TIMESTAMP_NTZ(9)	Y	default CURRENT_TIMESTAMP()	Last upsert time for this row

Declared key: PRIMARY KEY (CITY , DATE) (declarative only in Snowflake; enforced by the MERGE + duplicate-check in load() , not by the database).

DEV.ANALYTICS.stg_weather (dbt view)

Column	Type	Nullable	Description
CITY	VARCHAR(64)	N	Carried through from raw
LATITUDE	FLOAT	N	Carried through from raw
LONGITUDE	FLOAT	N	Carried through from raw
WEATHER_DATE	DATE	N	Renamed from raw's DATE
TEMP_MAX	FLOAT	Y	Carried through from raw
TEMP_MIN	FLOAT	Y	Carried through from raw
PRECIPITATION	FLOAT	Y	Carried through from raw
WEATHER_CODE	NUMBER(38,0)	Y	Carried through from raw
CITY_DATE	VARCHAR(16777216)	Y	Surrogate key: CITY '_' TO_VARCHAR(DATE , 'YYYY-MM-DD') ; tested unique + not_null in models/schema.yml despite being nullable at the database level

No declared key (view). CITY / LATITUDE / LONGITUDE / WEATHER_DATE inherit NOT NULL from the raw table's matching columns; CITY_DATE is a computed expression, which Snowflake always types as nullable regardless of its inputs.

DEV.ANALYTICS.weather_analytics (dbt table -- BI source)

Column	Type	Nullable	Description
CITY	VARCHAR(64)	Y	Partition key for every window function
LATITUDE	FLOAT	Y	Carried through
LONGITUDE	FLOAT	Y	Carried through
WEATHER_DATE	DATE	Y	Calendar date
CITY_DATE	VARCHAR(16777216)	Y	Surrogate key, tested <code>unique + not_null</code>
TEMP_MAX	FLOAT	Y	Carried through
TEMP_MIN	FLOAT	Y	Carried through
PRECIPITATION	FLOAT	Y	Carried through
WEATHER_CODE	NUMBER(38,0)	Y	Carried through
DAILY_AVG_TEMP	FLOAT	Y	$(TEMP_MAX + TEMP_MIN) / 2$
DRY_SPELL_LENGTH	NUMBER(18,0)	Y	Consecutive dry days (< 0.1mm precip) ending on this date
MOVING_AVG_TEMP_7D	FLOAT	Y	7-day trailing average of <code>DAILY_AVG_TEMP</code> , <code>PARTITION BY city</code> , rounded to 2 decimals
TEMP_ANOMALY	FLOAT	Y	<code>DAILY_AVG_TEMP</code> minus that city's average over the loaded window, rounded to 2 decimals
ROLLING_PRECIP_7D	FLOAT	Y	7-day trailing sum of <code>PRECIPITATION</code> , <code>PARTITION BY city</code> , rounded to 2 decimals

No declared key (table, materialized from a `SELECT`). Every column is nullable at the database level -- `CREATE TABLE AS SELECT` does not carry `NOT NULL` forward from `int_weather_metrics / stg_weather`, even though the `city_date / city / weather_date` columns are never actually null in practice. Uniqueness and non-null-ness are enforced by the dbt tests in `models/schema.yml`, not by the table's DDL.

DEV.SNAPSHOTS.weather_snapshot (dbt snapshot)

Column	Type	Nullable	Description
CITY	VARCHAR(64)	Y	From weather_analytics
LATITUDE	FLOAT	Y	From weather_analytics
LONGITUDE	FLOAT	Y	From weather_analytics
WEATHER_DATE	DATE	Y	From weather_analytics
CITY_DATE	VARCHAR(16777216)	Y	Snapshot unique_key
TEMP_MAX	FLOAT	Y	Tracked by check strategy
TEMP_MIN	FLOAT	Y	Tracked by check strategy
PRECIPITATION	FLOAT	Y	Tracked by check strategy
WEATHER_CODE	NUMBER(38,0)	Y	Tracked by check strategy
DAILY_AVG_TEMP	FLOAT	Y	From weather_analytics , not a check_cols column
DRY_SPELL_LENGTH	NUMBER(18,0)	Y	From weather_analytics , not a check_cols column
MOVING_AVG_TEMP_7D	FLOAT	Y	Tracked by check strategy
TEMP_ANOMALY	FLOAT	Y	Tracked by check strategy
ROLLING_PRECIP_7D	FLOAT	Y	Tracked by check strategy
DBT_SCD_ID	VARCHAR(32)	Y	dbt-generated: unique ID for this SCD Type 2 row version
DBT_UPDATED_AT	TIMESTAMP_NTZ(9)	Y	dbt-generated: when this row version was recorded
DBT_VALID_FROM	TIMESTAMP_NTZ(9)	Y	dbt-generated: start of this row version's validity
DBT_VALID_TO	TIMESTAMP_NTZ(9)	Y	dbt-generated: end of this row version's validity (NULL = current)

Snapshot key: `unique_key='city_date'` , `strategy='check'` on `[temp_max, temp_min, precipitation, weather_code, moving_avg_temp_7d, temp_anomaly, rolling_precip_7d]` (see `snapshots/weather_snapshot.sql`). `daily_avg_temp` and `dry_spell_length` are carried through but not in `check_cols` , so a change in only those two columns would not, by itself, open a new row version.

9. Idempotency Design

Rerunning `weather_etl_dag` for the same day must not create duplicate rows. This is enforced in `load()` (`days/weather_etl_dag.py`) as follows:

1. `CREATE TABLE IF NOT EXISTS` runs **before** `BEGIN` , since Snowflake DDL auto-commits and would otherwise break an open transaction.

2. New rows are inserted into a temporary staging table, then merged into `CITY_WEATHER_DAILY` with `MERGE ... ON (city, date) WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT -- all inside one BEGIN`.
3. Before `COMMIT`, a `GROUP BY (city, date) HAVING COUNT(*) > 1` query verifies no duplicate keys exist, because Snowflake's declared `PRIMARY KEY` is not enforced.
4. Any exception anywhere in the block triggers `ROLLBACK` and re-raises, so Airflow marks the task failed and no partial write is left committed.

Net effect: running the DAG twice for the same date updates that date's row in place rather than appending a second copy.

10. dbt Implementation

See `dbt/README.md` for the full layer-by-layer breakdown. Summary: `stg_weather` (view) -> `int_weather_metrics` (ephemeral) -> `weather_analytics` (table), all analytics window functions `PARTITION BY city`.

11. dbt Models

- `models/staging/stg_weather.sql`
- `models/intermediate/int_weather_metrics.sql`
- `models/analytics/weather_analytics.sql`

12. dbt Tests

Defined in `models/schema.yml`: `unique + not_null` on `city_date` for both `stg_weather` and `weather_analytics`; `not_null` on `city` and `weather_date`; `accepted_values` on `city` (Seoul, Toronto).

Chrome 피알 수정 보기 방문 기록 북마크 프로필 탭 창 도움말

weather_dbt_dag - Grid - Air X +

localhost:8082/dags/weather_dbt_dag/grid?tab=logs&dag_run_id=manual__2026-09-25T18%3A44%3A08.656060%2B00%3A00&task_id=dbt_run

Airflow DAGs Cluster Activity Datasets Security Browse Admin Docs

20:39 UTC

DAG: weather_dbt_dag dbt run -> dbt snapshot -> dbt test for the weather analytics models

Schedule: None Next Run ID: None

2026. 09. 25. 오후 08:38:40 All Run Types All Run States Clear Filters

Auto-refresh 25

Press **shift** + **/** for Shortcuts

deferred failed queued removed restarting running scheduled shutdown skipped success up_for_reschedule up_for_retry upstream_failed no_status

Duration 00:00:35 00:00:17 00:00:00

dbt_run dbt_snapshot dbt_test

DAG weather_dbt_dag Run Task 2026-09-25, 18:44:08 UTC dbt_run

Clear task Mark state as... Filter DAG by task

Details Graph Gantt Code Event Log Logs XCom Task Duration

All Levels All File Sources Wrap Download See More

```
4177b569b98c
*** Found local files:
*** * /opt/airflow/logs/dag_id=weather_dbt_dag/run_id=manual__2026-09-25T18:44:08.656060+00:00/task_id=dbt_run/attempt=1.log
[2026-09-25, 18:44:09 UTC] (local_task_job_runner.py:123) ▶ Pre task execution logs
[2026-09-25, 18:44:09 UTC] (subprocess.py:63) INFO - Tmp dir root location: /tmp
[2026-09-25, 18:44:09 UTC] (subprocess.py:75) INFO - Running command: ['/usr/bin/bash', '-c', 'cd /opt/***/dbt && dbt run --profiles-dir /opt/***/dbt']
[2026-09-25, 18:44:09 UTC] (subprocess.py:86) INFO - Output:
[2026-09-25, 18:44:10 UTC] (subprocess.py:93) INFO - 18:44:10 Running with dbt=1.8.7
[2026-09-25, 18:44:11 UTC] (subprocess.py:93) INFO - 18:44:11 Registered adapter: snowflake=1.8.1
[2026-09-25, 18:44:11 UTC] (subprocess.py:93) INFO - 18:44:11 Unable to do partial parsing because saved manifest not found. Starting full parse.
[2026-09-25, 18:44:12 UTC] (subprocess.py:93) INFO - 18:44:12 Found 3 models, 1 snapshot, 10 data tests, 1 source, 454 macros
[2026-09-25, 18:44:12 UTC] (subprocess.py:93) INFO - 18:44:12
[2026-09-25, 18:44:15 UTC] (subprocess.py:93) INFO - 18:44:15 Concurrency: 4 threads (target='dev')
[2026-09-25, 18:44:15 UTC] (subprocess.py:93) INFO - 18:44:15
[2026-09-25, 18:44:15 UTC] (subprocess.py:93) INFO - 18:44:15 1 of 2 START sql view model analytics.stg_weather ..... [RUN]
[2026-09-25, 18:44:16 UTC] (subprocess.py:93) INFO - 18:44:16 1 of 2 OK created sql view model analytics.stg_weather ..... [SUCCESS 1 in 1.33s]
[2026-09-25, 18:44:16 UTC] (subprocess.py:93) INFO - 18:44:16 2 of 2 START sql table model analytics.weather_analytics ..... [RUN]
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19 2 of 2 OK created sql table model analytics.weather_analytics ..... [SUCCESS 1 in 2.35s]
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19 Finished running 1 view model, 1 table model in 0 hours 0 minutes and 6.57 seconds (6.57s).
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19 Completed successfully
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19
[2026-09-25, 18:44:19 UTC] (subprocess.py:93) INFO - 18:44:19 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
[2026-09-25, 18:44:20 UTC] (subprocess.py:97) INFO - Command exited with return code 0
[2026-09-25, 18:44:20 UTC] (taskinstance.py:340) ▶ Post task execution logs
```

Chrome 피알 수정 보기 방문 기록 북마크 프로필 탭 창 도움말

weather_dbt_dag - Grid - Air X +

localhost:8082/dags/weather_dbt_dag/grid?tab=logs&dag_run_id=manual__2026-09-25T18%3A44%3A08.656060%2B00%3A00&task_id=dbt_test

Airflow DAGs Cluster Activity Datasets Security Browse Admin Docs

20:40 UTC

DAG: weather_dbt_dag dbt run -> dbt snapshot -> dbt test for the weather analytics models

Schedule: None Next Run ID: None

2026. 09. 25. 오후 08:38:40 All Run Types All Run States Clear Filters

Auto-refresh 25

Press **shift** + **/** for Shortcuts

deferred failed queued removed restarting running scheduled shutdown skipped success up_for_reschedule up_for_retry upstream_failed no_status

Duration 00:00:35 00:00:17 00:00:00

dbt_run dbt_snapshot dbt_test

DAG weather_dbt_dag Run Task 2026-09-25, 18:44:08 UTC dbt_test

Clear task Mark state as... Filter DAG by task

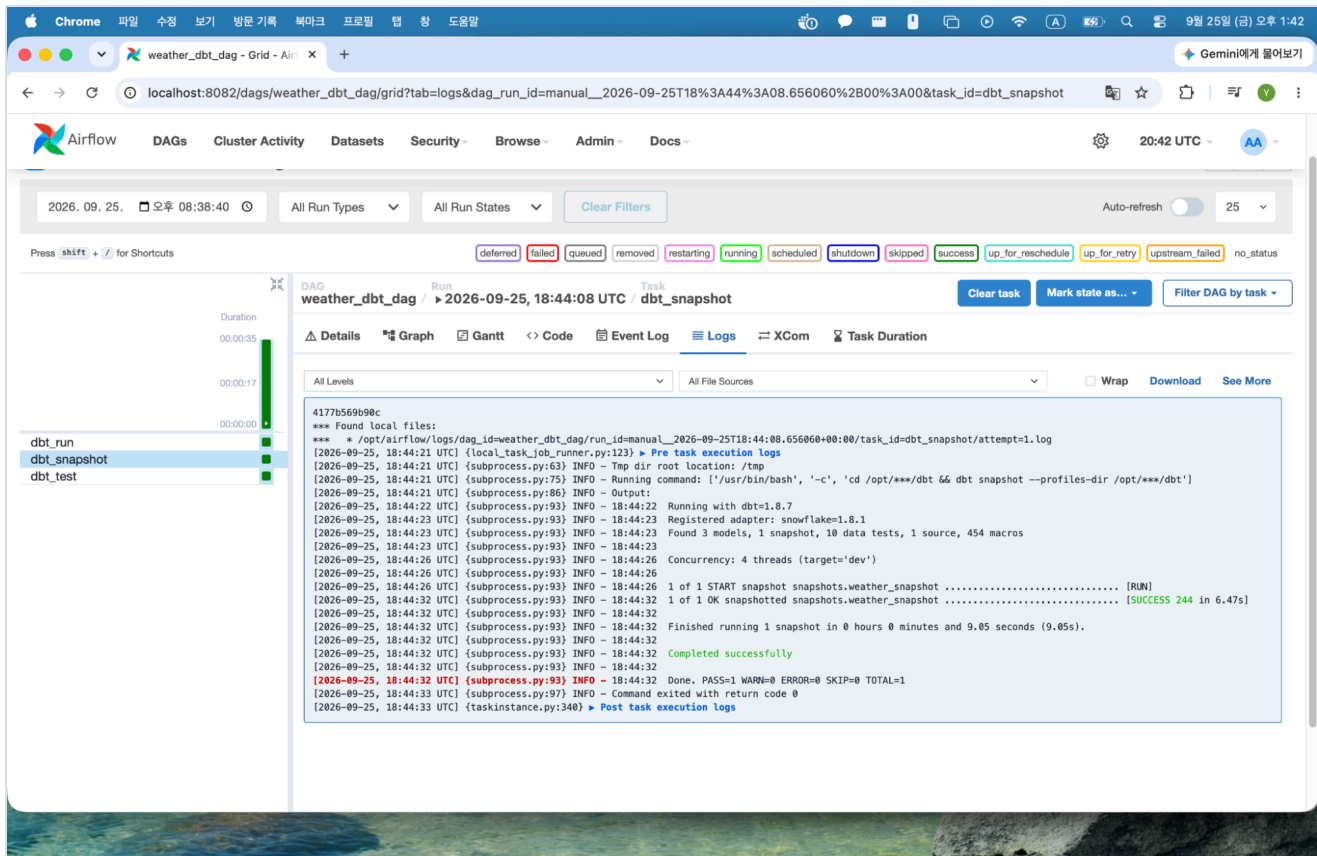
Details Graph Gantt Code Event Log Logs XCom Task Duration

All Levels All File Sources Wrap Download See More

```
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38 Concurrency: 4 threads (target='dev')
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38 1 of 10 START test accepted_values_stg_weather_city_Seoul_Toronto ..... [RUN]
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38 2 of 10 START test accepted_values_weather_analytics_city_Seoul_Toronto ..... [RUN]
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38 3 of 10 START test not_null_stg_weather_city ..... [RUN]
[2026-09-25, 18:44:38 UTC] (subprocess.py:93) INFO - 18:44:38 4 of 10 START test not_null_stg_weather_city_date ..... [RUN]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 1 of 10 PASS accepted_values_stg_weather_city_Seoul_Toronto ..... [PASS in 1.32s]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 3 of 10 PASS not_null_stg_weather_city ..... [PASS in 1.32s]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 5 of 10 START test not_null_stg_weather_weather_date ..... [RUN]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 6 of 10 START test not_null_weather_analytics_city ..... [PASS in 1.34s]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 7 of 10 START test not_null_weather_analytics_city_date ..... [RUN]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 4 of 10 PASS not_null_stg_weather_city_date ..... [PASS in 1.52s]
[2026-09-25, 18:44:40 UTC] (subprocess.py:93) INFO - 18:44:40 8 of 10 START test not_null_weather_analytics_weather_date ..... [RUN]
[2026-09-25, 18:44:41 UTC] (subprocess.py:93) INFO - 18:44:41 5 of 10 PASS not_null_stg_weather_weather_date ..... [PASS in 1.14s]
[2026-09-25, 18:44:41 UTC] (subprocess.py:93) INFO - 18:44:41 9 of 10 START test unique_stg_weather_city_date ..... [PASS in 1.15s]
[2026-09-25, 18:44:41 UTC] (subprocess.py:93) INFO - 18:44:41 10 of 10 START test unique_weather_analytics_city_date ..... [RUN]
[2026-09-25, 18:44:41 UTC] (subprocess.py:93) INFO - 18:44:41 8 of 10 PASS not_null_weather_analytics_weather_date ..... [PASS in 1.09s]
[2026-09-25, 18:44:41 UTC] (subprocess.py:93) INFO - 18:44:41 7 of 10 PASS not_null_weather_analytics_city_date ..... [PASS in 1.08s]
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42 9 of 10 PASS unique_stg_weather_city_date ..... [PASS in 1.17s]
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42 10 of 10 PASS unique_weather_analytics_city_date ..... [PASS in 1.38s]
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42 Finished running 10 data tests in 0 hours 0 minutes and 5.23 seconds (5.23s).
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42 Completed successfully
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42
[2026-09-25, 18:44:42 UTC] (subprocess.py:93) INFO - 18:44:42 Done. PASS=10 WARN=0 ERROR=0 SKIP=0 TOTAL=10
[2026-09-25, 18:44:43 UTC] (subprocess.py:97) INFO - Command exited with return code 0
[2026-09-25, 18:44:43 UTC] (taskinstance.py:340) ▶ Post task execution logs
```

13. dbt Snapshot

`snapshots/weather_snapshot.sql`, `strategy='check'`, keyed on `city_date`. See Section 8 for its resulting columns and Section 3 for why `check` was chosen over `timestamp`.



The screenshot shows the Apache Airflow web interface. The top navigation bar includes links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The main content area displays the execution details for the `weather_dbt_dag` DAG, specifically the `dbt_snapshot` task. The task is shown as completed successfully. The logs indicate that the task ran on 2026-09-25 at 18:44:08 UTC. The logs show the task's execution steps, including finding local files, running the dbt command, and successfully completing the snapshot. The task duration is 00:00:35.

14. Airflow/dbt Scheduling

`weather_etl_dag` is the only DAG on a cron schedule (`30 2 * * *`). `weather_dbt_dag` has `schedule=None` and is started exclusively by `weather_etl_dag`'s last task, `trigger_dbt_dag` (`TriggerDagRunOperator(trigger_dag_id="weather_dbt_dag")`). This makes the "ETL succeeds, then dbt runs" dependency visible directly in the DAG code and in the Airflow UI (two separate DAGs, one triggering the other), rather than relying on scheduling two DAGs far enough apart in time.

15. BI Dashboard

Status: complete. **Tool:** [Preset](#) (cloud-hosted Apache Superset). **Team/Workspace:** "DATA 226 Weather Analytics" / "Weather Analytics Lab". **Dashboard name:** "Weather Analytics Dashboard". **Dataset:** `DEV.ANALYTICS.WEATHER_ANALYTICS`.

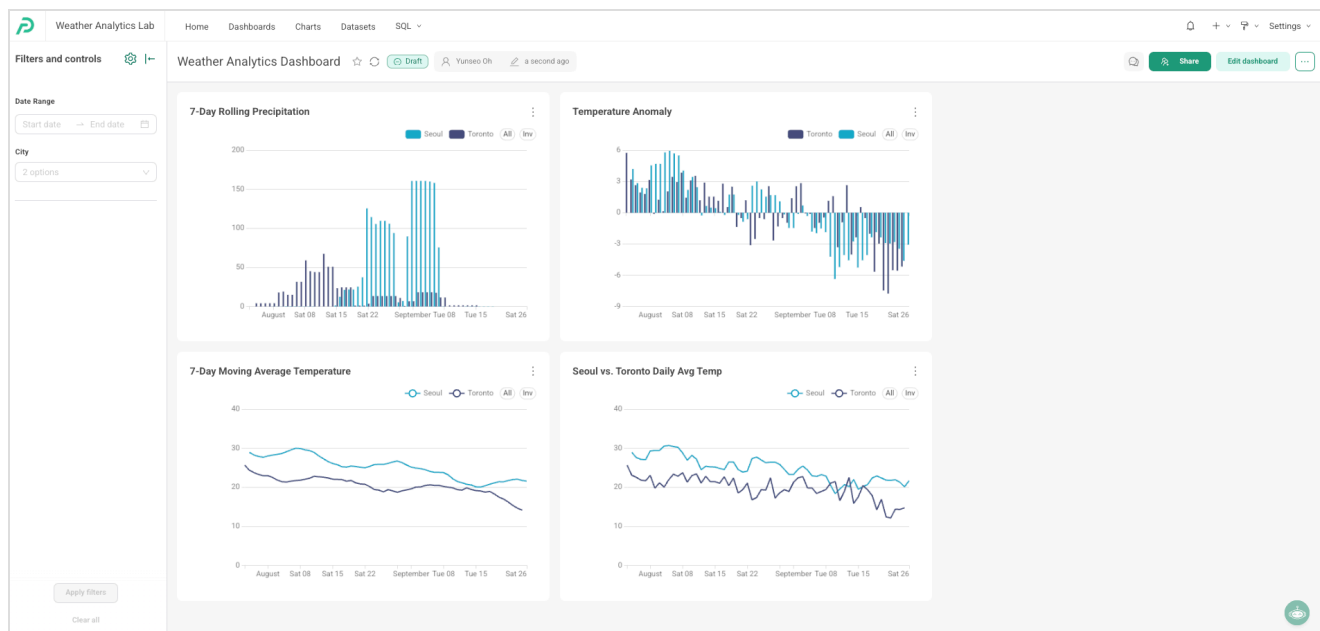
Purpose: let a viewer compare Seoul vs. Toronto's current weather trend -- temperature level and trajectory, anomaly, and precipitation -- without reading raw numbers. **Usage:** select a city (or leave both selected) via the dashboard's `CITY` filter, narrow the `WEATHER_DATE` time range, and all four charts update together.

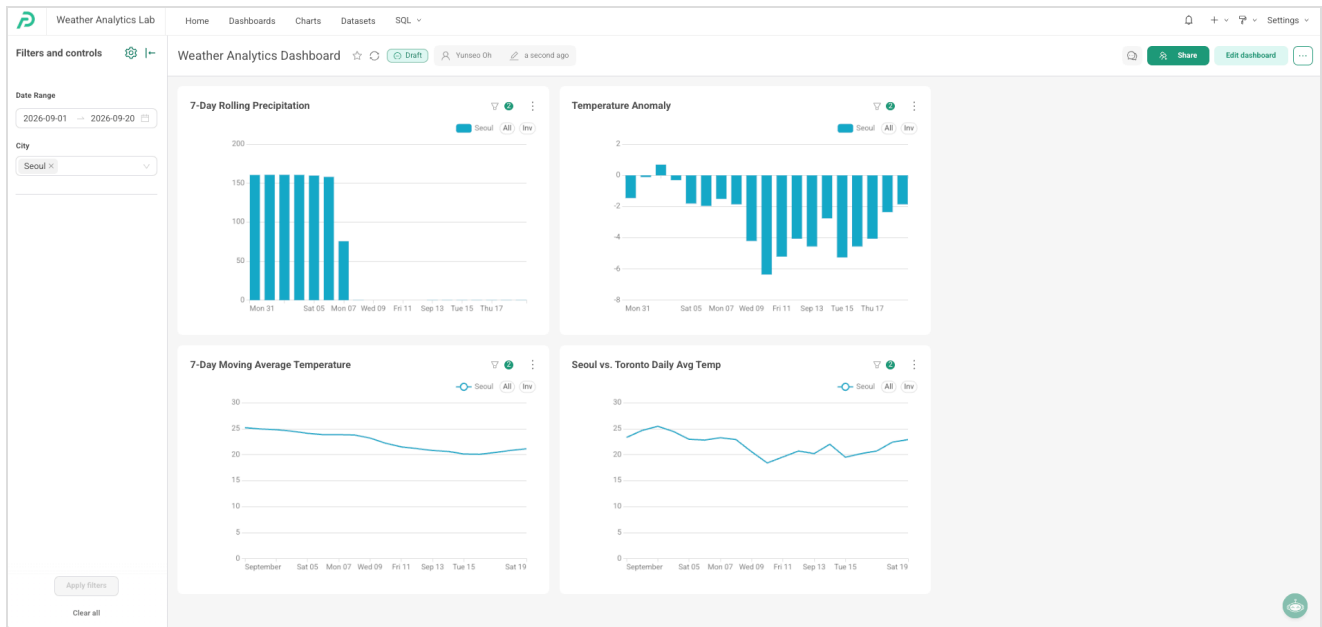
Authentication: Preset could not use a plain Snowflake username/password connection because this account requires MFA/TOTP. The dashboard instead connects using the same Snowflake **key-pair authentication** already used by Airflow and dbt, configured through Preset's **Advanced -> Security -> Secure Extra** field rather than the basic login form. That field lives only in Preset's hosted connection config -- the key and passphrase were never written to this repository. See the root `README.md` -> "Security / secrets".

The four charts, all grouped by `CITY` :

1. **Seoul vs. Toronto Daily Avg Temp** -- line chart, X = `WEATHER_DATE` , metric `AVG(DAILY_AVG_TEMP)` , dimension `CITY` .
2. **7-Day Moving Average Temperature** -- line chart, X = `WEATHER_DATE` , metric `AVG(MOVING_AVG_TEMP_7D)` , dimension `CITY` .
3. **Temperature Anomaly** -- bar chart, X = `WEATHER_DATE` , metric `AVG(TEMP_ANOMALY)` , dimension `CITY` .
4. **7-Day Rolling Precipitation** -- bar chart, X = `WEATHER_DATE` , metric `AVG(ROLLING_PRECIP_7D)` , dimension `CITY` .

Filters: native dashboard filters on `CITY` and on `WEATHER_DATE` (time range), applied across all four charts at once.





16. Results / Observations

The pipeline was run end-to-end against live data (see Section 18, Conclusion, for the full verification summary). As of the run on 2026-09-25/26, over the ~61-day loaded window:

- Both cities were in the middle of an extended dry spell at the same time: Seoul had gone 14 consecutive days with under 0.1mm of precipitation, Toronto 17 days -- both `rolling_precip_7d` values were at or effectively at 0.
- Both cities' most recent days ran cooler than their loaded-window average: Seoul's temperature anomaly was around -3 to -5 degrees C over its last several days; Toronto's was around -5 to -8 degrees C, reaching -7.76 on 2026-09-22 (its `daily_avg_temp` that day, 12.15 degrees C, was well under its ~19.9 degrees C window average).
- Seoul's window average `daily_avg_temp` (~24.8 degrees C) was meaningfully warmer than Toronto's (~19.9 degrees C) over the same calendar period, as expected in late-summer-to-fall for the two cities' respective climates.
- Anomaly range across the window: Seoul -6.36 to +5.94 degrees C; Toronto -7.76 to +5.74 degrees C -- both cities show comparable volatility around their own baseline despite the ~5 degree gap in absolute average temperature, which is exactly the kind of city-relative comparison `temp_anomaly` (computed `PARTITION BY city`) is meant to surface.

These figures came from querying `DEV.ANALYTICS.WEATHER_ANALYTICS` directly after a real `dbt` run , and the Preset dashboard confirms the same pattern visually (Figures 9-10, Section 15): the default-view "Temperature Anomaly" chart ranges from roughly -8 to +6 for both cities, matching the -7.76 to +5.94 range computed here, and "7-Day Rolling Precipitation" shows both cities' bars taper toward zero at the right edge of the window, matching the Seoul/Toronto dry spell described above. The filtered view (Figure 10, isolated to Seoul, 2026-09-01 to 2026-09-20) shows Seoul's rolling precipitation dropping from roughly 150mm to near 0 by the end of that range and its anomaly staying mostly negative through September -- the same transition into the dry spell visible in the unfiltered chart, now isolated to one city.

17. Future Work

- Add more cities by extending the `CITY_CONFIG` Airflow Variable only.
- Backfill a longer history (Open-Meteo's archive API) to make `temp_anomaly` a true climatological anomaly instead of a within-loaded-window baseline.
- Add a `freshness` check on the `raw.city_weather_daily` source so a stale load surfaces before dbt silently reruns on old data.
- Alerting (Slack/email) on Airflow task failure or dbt test failure.

18. Conclusion

The pipeline was verified end-to-end in a real local Airflow environment: `weather_etl_dag` (extract, transform, load) and the triggered `weather_dbt_dag` (dbt run, snapshot, test) both completed successfully against the live Snowflake account, loading real Seoul and Toronto data and passing all 10 dbt tests. Rerunning the ETL DAG confirmed the MERGE-based load is idempotent (stable per-city row counts, zero duplicate keys). One Airflow codebase and one dbt project handle both cities through the `CITY_CONFIG` Variable and `PARTITION BY city` window functions, satisfying the assignment's requirement to treat city as configuration rather than forking the pipeline per city.

The BI layer is also complete: a Preset dashboard ("Weather Analytics Dashboard") connects to `DEV.ANALYTICS.WEATHER_ANALYTICS` -- via Snowflake key-pair authentication through Preset's Secure Extra configuration, working around this account's MFA requirement -- and presents four charts (daily average temperature, 7-day moving average, temperature anomaly, 7-day rolling precipitation), all grouped by city and filterable by city and date range. End to end, the project satisfies Open-Meteo -> Airflow -> Snowflake RAW -> dbt -> Snowflake analytics -> BI dashboard with one shared pipeline for both cities, and all 10 required screenshots (Airflow Variables and Connections, both DAG graph views, the ETL load log, the three dbt task logs, and both dashboard views) are captured and present in `screenshots/`, embedded throughout this report as Figures 1-10 (Sections 6, 12, 13, and 15), alongside the system architecture diagram in Section 4. The repository is public at the URL above, and the submission date is recorded above. This report is complete.

19. References

- [Open-Meteo Forecast API](#)
- [Snowflake key-pair authentication](#)
- [Snowflake transactions](#)
- [Snowflake MERGE](#)
- [Apache Airflow Snowflake provider](#)
- [Apache Airflow TriggerDagRunOperator](#)
- [dbt snapshots](#)
- [dbt tests](#)
- Course materials: SJSU DATA 226, Week 3 (Data Pipelines & Airflow), Week 4 (Advanced Airflow), Week 5 (ELT & dbt) -- Keeyong Han

- This team's own HW2 (Open-Meteo -> Snowflake), HW3 (`weather_to_snowflake_airflow.py`), and HW4 (`data226-hw4-dbt`), whose transaction/idempotency and dbt-layering patterns this project builds on